

AI Football Scouting Assistant -- Project Case Study

Серверлесс AI-платформа аналитики: детерминированный стат-движок в Python + ReAct LLM-оркестрация

GitHub Repository: <https://github.com/Creeepling/ai-football-scouting-bot>

Стек технологий: Python 3.11, Google Cloud Functions (Gen 2), LangChain, Wyscout API, MongoDB Atlas, Cloud Storage, Matplotlib, Telegram API

1. Архитектурная философия и решаемая проблема

Архитектурный принцип: Полное разделение математических вычислений и языковой генерации. Статистика, Per-90, перцентили и Байесовские сглаживания рассчитываются детерминированно в Python/Pandas. LLM не является источником числовой истины и используется исключительно для распознавания сущностей (Entity Resolution), оркестрации инструментов (ReAct Tool Calling) и контекстной тактической интерпретации.

Проблема: Скаутинг требует ручного сопоставления профилей Transfermarkt и Wyscout, вычисления перцентилей относительно лиги/позиции, построения графиков и написания резюме. Продукт автоматизирует этот цикл от ссылки до готового досье со скаутинг-картой и экспертным резюме в Telegram.

2. Кто пользователь

- Спортивные директоры и селекционные отделы футбольных клубов.
- Футбольные скауты и матчевые аналитики, проводящие первичный скрининг игроков.
- Футбольные агентства для подготовки аналитических досье и презентаций кандидатов.

3. Что именно сделано лично (Engineering Ownership)

- Архитектура и Backend: Спроектировал и реализовал модульный серверлесс-сервис на Python с изоляцией сессий через contextvars для потокобезопасной обработки запросов.
- Детерминированный стат-движок: Разработал формулы нормализации показателей: Per-90, перцентильные ранги по лигам и Байесовское сглаживание (bayes_rank) для метрик с малым числом попыток.
- Data Pipelines & Агрегации: Написал коннекторы к Wyscout API v3 и MongoDB Atlas со сложными агрегационными пайплайнами сезонной и матчевой статистики.
- AI Agent & Tool Calling: Построил ReAct-агента на базе LangChain с набором инструментов (поиск сущностей, выбор сезона, глубокий анализ метрик и расчет игровых позиций).
- Генератор графических отчетов: Реализовал рендеринг скаутинг-карт, гистограмм распределения, тепловых карт и круговых диаграмм на Matplotlib/Seaborn с контролем утечек памяти.
- DevOps & Безопасность: Настроил zero-secret архитектуру через GCP Secret Manager и автоматизировал CI/CD деплой в GitHub Actions через Workload Identity Federation (WIF).

4. Архитектура системы

Поток данных: Telegram Webhook -> GCP Cloud Function (Router) -> Thread-Safe Context (contextvars) -> ReAct Agent (LangChain) -> Data Layer (Wyscout API, MongoDB Atlas, Serper API) -> Deterministic Engine (Pandas / NumPy) -> Visualization Engine (Matplotlib / Seaborn) -> GCS Session Store -> Telegram Bot API.

5. Используемый стек, API и источники данных

- Core & Serverless: Python 3.11, Google Cloud Functions (Gen 2 / Cloud Run), Flask / Functions Framework, PyMongo, Pydantic.
- AI / LLM & Оркестрация: LangChain, Google Gemini API / OpenAI (gemini-3-flash-preview), Google Serper API.
- Базы данных и хранилище: MongoDB Atlas (событийная статистика матчей), Google Cloud Storage (JSON-сессии и справочники метрик).
- Data Science & Визуализация: Pandas, NumPy, Matplotlib, Seaborn, Pillow.
- Внешние сервисы: Wyscout API v3 (профили, карьеры, сезоны, соревнования), Telegram Bot API.

6. Где и как используется AI / LLM (Tool Calling)

- Fuzzy Entity Resolution: Парсинг неструктурированных ссылок Transfermarkt через Serper + LLM-экстрактор и точный маппинг на базу Wyscout.
- ReAct Tool Orchestration: Автономный выбор и вызов аналитических инструментов агентом в зависимости от контекста диалога (выбор сезона, срез метрики, тепловая карта, распределение позиций).
- Tactical Scouting Synthesis: Генерация экспертного резюме на базе перцентилей игрока (оценка баланса качеств, выявление аномалий и сильных сторон на фоне конкурентов по лиге).

7. Развертывание и инфраструктура (DevOps)

Сервис развернут в бессерверной среде Google Cloud Functions (Gen 2) с масштабированием в 0. Конфиденциальные ключи изолированы в GCP Secret Manager и динамически монтируются при запуске. Деплой автоматизирован через GitHub Actions с беспарольной аутентификацией через Workload Identity Federation.

8. Что было сложным технически (Key Technical Challenges)

- Small-Sample Variance: Игрок с 2 успешными обводками из 2 получал 100-й перцентиль. Решено внедрением Байесовского сглаживания с учетом априорного веса 80-го перцентиля лиги.
- Fuzzy Entity Matching: Сопоставление локализованных имен и транслитераций Transfermarkt с жестким поисковым индексом Wyscout.

- Matplotlib в Serverless: Устранение утечек памяти в переиспользуемых контейнерах Cloud Functions через строгий вызов `plt.close(fig)` и бинарные потоки `io.BytesIO`.
- Потокбезопасность: Изоляция контекста запросов (`chat_id`, язык) без глобальных переменных через `contextvars`.

9. Что бы сегодня сделал иначе (Future Evolution)

- Асинхронные очереди (Cloud Tasks / Pub/Sub): Вынесение генерации тяжелых графиков в фоновые воркеры с пуш-уведомлением для исключения рисков вебхук-таймаута Telegram.
- Similarity Search (Поиск аналогов): Добавление векторного поиска (Vector DB / pgvector) по вектору перцентилей для автоматического подбора похожих игроков (Player Comparison).
- Кэширование (Redis): Кэширование профилей и агрегированных срезов Wyscout для минимизации расходов на вызовы сторонних API.
- Modern LCEL & Structured Output: Переход на Pydantic Function Calling в LangChain вместо парсинга строк ReAct.